

Guia de Documentação do Projeto

Simulador de Escalonamento de Tarefas (Python)

Sistemas Operacionais

Apresentação

Este guia orienta o conteúdo esperado em cada documento da entrega do projeto. A entrega é composta por três tipos de documento, cada um com um propósito específico:

Documento	Pergunta que responde	Local
README	"O que é isso e por onde começo?"	Raiz do repositório
Tutoriais	"Como eu executo e uso isso?"	<code>docs/</code>
Documentação	"Como isso funciona internamente?"	<code>docs/</code>

A documentação é parte da entrega, não um acessório dela. O projeto é avaliado por quem **não** participou do desenvolvimento e abrirá o programa com dois cliques, sem montar ambiente nem digitar comando algum. Tudo o que for necessário para usar e entender o simulador precisa estar escrito.

1 README

O README é o ponto de entrada do repositório. É o primeiro arquivo lido por quem chega ao projeto e deve dar uma visão geral rápida do que está ali e de como navegar. Deve conter, no mínimo:

- Título do projeto;
- Disciplina, semestre e nome do(a) professor(a);
- Autoria;
- Breve descrição do projeto (1 a 2 parágrafos);
- **Como executar:** qual arquivo abrir com dois cliques. Esta informação vem antes de qualquer outra, porque é a primeira coisa que quem avalia vai procurar;
- **Estrutura do repositório:** árvore de pastas explicando o que há em cada lugar;
- Lista dos arquivos de código com explicação breve do que cada um faz;
- Requisitos de ambiente: versão do Python e bibliotecas utilizadas, se houver;
- **Funcionalidades:** lista do que o simulador faz, com indicação do arquivo em que cada funcionalidade está implementada;
- Atalhos para os tutoriais e para a documentação dentro de `docs/`;
- **Por onde começar:** trilha de leitura indicando a ordem dos documentos.

Exemplo de estrutura do README

```
# Simulador de Escalonamento de Tarefas

Projeto pratico da disciplina de Sistemas Operacionais,
ministrada por <nome do professor>. Semestre <ano/semestre>.

## Autoria
- Nome

## Descricao
Simulador de escalonamento de tarefas em um processador.
Implementa seis algoritmos (FCFS, SJF, SRTF, Round-Robin e
prioridade cooperativa e preemptiva), trata recursos de uso
exclusivo e reproduz o fenomeno da inversao de prioridades,
com os mecanismos de heranca e teto de prioridade.

## Como executar
Clique duas vezes em 'Simulador.exe'.
Nao e necessario instalar nada.

## Estrutura do repositorio

'''
simulador_escalamento/
|-- Simulador.exe      Programa pronto para executar
|-- main.py           Ponto de entrada do codigo-fonte
|-- simulador/        Codigo-fonte do simulador
|-- cenarios/         Conjuntos de tarefas em JSON
'-- docs/             Tutoriais e documentacao tecnica
'''

## Arquivos de codigo
- 'simulador/modelo.py' - estrutura de uma tarefa
- 'simulador/motor.py'  - laco de simulacao (mecanismo)
- 'simulador/politicas.py' - os seis algoritmos (politica)
- 'simulador/metricas.py' - calculo de tt, tp e tw
- 'simulador/interface.py' - janela do programa

## Funcionalidades
| O que faz | Onde |
|-----|-----|
| Os seis algoritmos | 'simulador/politicas.py' |
| Metricas por tarefa | 'simulador/metricas.py' |
| Recurso exclusivo | 'simulador/motor.py' |
| Heranca e teto | 'simulador/motor.py' |
| Sorteio de cenarios | 'simulador/gerador.py' |

## Documentacao
- [Tutorial de execucao](./docs/tutorial_execucao.pdf)
- [Tutorial de uso](./docs/tutorial_uso.pdf)
- [Documentacao tecnica](./docs/documentacao_projeto.pdf)

## Por onde comecar
1. Abra o programa e siga o tutorial de execucao
```

2. Reproduza um cenário de exemplo pelo tutorial de uso
3. Consulte a documentação técnica para entender o código

2 Tutoriais (docs/tutorial_*.pdf)

Os tutoriais são documentos **operacionais**: explicam **como fazer**, passo a passo. Não precisam justificar decisões nem explicar conceitos, apenas guiar a execução. Devem ser autoexplicativos a ponto de outra pessoa conseguir usar o simulador sem precisar perguntar nada.

2.1 tutorial_execucao.pdf

Como colocar o programa para funcionar. É o documento mais curto e o mais importante: se ele falhar, nada mais será avaliado.

Roteiro mínimo do tutorial de execução

1. **Pré-requisitos:** sistema operacional e, caso a entrega não seja um executável independente, versão do Python e bibliotecas necessárias;
2. **Abertura:** qual arquivo receber o duplo clique, com captura de tela da pasta descompactada;
3. **Primeira tela:** captura da janela inicial, identificando cada campo e cada botão;
4. **Execução mínima:** a sequência mais curta possível que produz um resultado na tela, para confirmar que o programa funciona;
5. **Resultado esperado:** captura do que deve aparecer ao final dessa sequência;
6. **Problemas conhecidos:** o que fazer se a janela não abrir ou fechar sozinha.

2.2 tutorial_uso.pdf

Como operar o simulador em cada uma de suas funções. Este tutorial descreve o programa do ponto de vista de quem o usa, e não de quem o escreveu.

Roteiro mínimo do tutorial de uso

1. **Definir o conjunto de tarefas:** escolher a quantidade e informar instante de ingresso, tempo de processamento t_p e prioridade de cada uma;
2. **Sortear tarefas:** como pedir um conjunto aleatório em vez de digitar;
3. **Definir os parâmetros de tempo:** onde se informa o quantum t_q e o custo da troca de contexto t_{tc} , e o que acontece ao informar um quantum menor que o custo;
4. **Escolher o algoritmo:** onde se seleciona entre os seis algoritmos e como habilitar herança, teto e envelhecimento;
5. **Declarar o uso do recurso:** como indicar que uma tarefa disputa o recurso R , informando após quanto tempo de processamento ela o **obtem** e por quanto tempo o **mantém**;
6. **Ler os resultados:** como interpretar a tabela de métricas e o diagrama de tempo, identificando os intervalos de execução, as trocas de contexto e os períodos em que uma tarefa fica **suspensa** à espera de R ;

7. **Gravar e recarregar um cenário:** como preservar um conjunto de tarefas para reexaminá-lo depois;
8. **Conferir um resultado conhecido:** um conjunto de tarefas de exemplo, com os valores que o simulador deve produzir, de modo que o leitor confirme que o programa está funcionando corretamente na máquina dele.

Observação importante: tutoriais devem ter **capturas de tela** ilustrando cada etapa. Texto puro não funciona bem como tutorial.

Exemplo de passo de tutorial

*"No campo **Número de tarefas**, informe 5 e pressione **Aplicar**. A tabela passa a exibir cinco linhas. Preencha a primeira linha com ingresso 0, tempo de processamento 5 e prioridade 2, conforme a figura 3. Repita para as demais tarefas com os valores da tabela 1."*

3 Documentação do Projeto

Arquivo docs/documentacao_projeto.pdf.

A documentação descreve **como o projeto funciona internamente**. É o documento que alguém leria para entender, modificar ou estender o simulador. Diferente do tutorial, aqui o foco está em **conceito** e não em execução.

Conteúdo esperado

1. **Visão geral:** o que o simulador faz e qual modelo de escalonamento implementa;
2. **Separação entre política e mecanismo:** onde termina o laço de simulação, comum a todos os algoritmos, e onde começa a decisão específica de cada um. Este é o eixo do projeto e merece seção própria;
3. **Diagrama de módulos:** representação visual dos componentes e do fluxo de dados, da entrada das tarefas até a apresentação das métricas e do diagrama de tempo;
4. **Estrutura de uma tarefa:** tabela com os campos que descrevem uma tarefa, seus tipos e funções. Exemplo:

Campo	Tipo	Função
id	inteiro	Identificador da tarefa
ingresso	inteiro	Instante em que a tarefa surge
tp	inteiro	Tempo de processamento demandado
prioridade	inteiro	Maior valor, prioridade mais alta
uso_de_r	tupla	Quando a tarefa obtém R e por quanto tempo o mantém

5. **Interface dos módulos:** para cada função ou classe principal, o que recebe, o que devolve e o que faz;
6. **Parâmetros configuráveis:** quantum t_q , custo da troca de contexto t_{tc} , passo do envelhecimento α e escolha do mecanismo de correção, indicando faixa válida e valor padrão de cada um;
7. **Funcionamento interno:** o que acontece a cada unidade de tempo do laço de simulação. Como o conjunto de tarefas prontas é montado, quando a troca de contexto é cobrada, como a tarefa que solicita um recurso ocupado se suspende e sai do conjunto de prontas, e como a prioridade efetiva é recalculada sob herança, teto e envelhecimento;
8. **Convenções de simulação:** as decisões de modelagem que determinam os números produzidos, sem as quais o mesmo conjunto de tarefas gera resultados diferentes. Entre elas: a unidade de tempo adotada, o critério de desempate entre tarefas equivalentes, em que momento a troca de contexto é cobrada e se o seu custo é descontado da fatia ou somado a ela, a posição em que uma tarefa preemptada retorna à fila, o sentido da escala de prioridades e a referência de tempo usada para situar o uso do recurso;
9. **Dependências e ambiente:** versão do Python, bibliotecas utilizadas e como o executável foi gerado.

Tom de escrita: a documentação é escrita no **presente** e de forma **impessoal**, como uma referência técnica.

Exemplo de trecho da documentação

"A cada unidade de tempo, o motor monta o conjunto de tarefas prontas com aquelas que já ingressaram, ainda não concluíram e não estão suspensas à espera do recurso. Esse conjunto é entregue à política, que devolve a tarefa escolhida. O motor não conhece o critério de escolha, e a política não conhece o relógio."

4 Comparativo entre os documentos

A tabela abaixo resume as principais diferenças entre os três tipos de documento, para evitar confusão sobre o que colocar em cada lugar:

	README	Tutorial	Documentação
Propósito	Orientação e navegação	Reprodução passo a passo	Referência técnica
Público	Quem chega ao projeto	Quem vai usar	Quem vai modificar
Responde	O que há aqui	Como eu faço	Como funciona
Tempo verbal	Presente	Imperativo	Presente
Pessoa	Impessoal	Impessoal	Impessoal
Tamanho	Curto	Médio	Médio
Imagens	Não	Capturas de tela	Diagramas
Atualiza com o código?	Sim	Sim	Sim

Resumo rápido

- **README** = "Estou no projeto. O que tem aqui e por onde começo?"
- **Tutorial** = "Como eu executo e uso isso passo a passo?"
- **Documentação** = "Como este simulador funciona internamente?"

Dicas finais

- Comece pelo README, que é curto e organiza o resto;
- Escreva o tutorial de uso **enquanto** opera o programa, não depois. Quem escreve de memória omite justamente os passos que se tornaram automáticos;
- A documentação pode ser construída em paralelo ao desenvolvimento do código;
- Capture as telas conforme o programa fica pronto, para não precisar refazer o percurso inteiro no fim;
- Teste a documentação como quem avalia fará: descompacte a entrega em uma pasta nova, siga o próprio tutorial ao pé da letra e veja se o programa abre e produz os números esperados;
- Releia tudo no final pensando: *"alguém que nunca viu este projeto entenderia isso sem perguntar nada?"*